

Audit complet detaille du site FSVB Studio

- Date : 2026-04-06
- Application auditee : fsvb-studio-portfolio/portfolio-app
- Base de mesure performance : build de production locale sur `http://127.0.0.1:4174/`
- Nature du livrable : second audit consolide detaille
- Methode : lecture du code, build production, lint, analyse des rapports Lighthouse, mobilisation des agents specialises du depot

1. Resume decisionnel

Verdict global

Le site est **montrable et deja convaincant sur la partie vitrine**, mais il reste **fragile sur plusieurs parcours qui comptent pour le business**.

En bref :

- la vitrine publique est globalement correcte
- les performances sont suffisantes pour une premiere impression, mais freinees par un chargement initial trop lourd
- le referencement reste trop basique pour bien soutenir l'acquisition
- certains parcours de confiance peuvent donner un faux sentiment de succes ou perdre un contact
- le configurateur concentre une grande partie de la complexite, de la dette technique et des causes possibles de regression

Niveau de priorite

1. Priorite haute

fiabiliser les parcours de conversion et de confiance

1. Priorite moyenne

allger l'application et renforcer les bases SEO

1. Priorite continue

reduire la dette technique du configurateur, des formulaires et des parcours admin

2. Perimetre audite

Pages auditees en runtime

- /
- /solutions
- /configurator
- /about
- /contact
- /templates/landing-solo
- /templates/restaurant
- /templates/coach
- /admin/login

Parcours revus surtout par lecture du code

- /confirmation
- /order/:id
- /form/:templateType
- /form/:templateType/:sessionId
- /configurator/session/:sessionId
- /admin/dashboard

Angles d'analyse couverts

- coordination generale
- audit UX
- revue design
- test de parcours utilisateur
- verification qualite
- investigation performance
- audit accessibilite
- revue SEO

- revue securite
- synthese documentaire

Limites connues

- pas de session admin reelle fournie
- pas de commande reelle fournie pour `/order/:id`
- pas de paiement reel fourni
- pas de session pre-session reelle fournie en base
- pas de session collaborative reelle fournie

3. Tableau performance par page

Scores Lighthouse de production locale, profil mobile.

Page	Performance	Accessibilite	Bonnes pratiques	SEO	FCP	LCP	TBT	CLS
---	---	---	---	---	---	---	---	---
Accueil	82 98 100	83	3.0 s 4.0 s 20 ms	0.011				
Solutions	83 98 100	83	3.0 s 3.8 s 10 ms	0.003				
Configurateur	74 94 100	83	3.0 s 5.4 s 100 ms	0.017				
A propos	75 98 100	83	3.0 s 5.3 s 0 ms	0.000				
Contact	83 98 100	82	3.0 s 3.8 s 0 ms	0.001				
Template landing-solo	74 92 100	83	3.0 s 5.7 s 40 ms	0.013				
Template restaurant	83 94 100	83	3.0 s 3.9 s 30 ms	0.012				
Template coach	78 98 100	83	3.0 s 4.3 s 30 ms	0.084				
Admin login	83 88 100	82	3.0 s 3.8 s 10 ms	0.001				

Lecture simple

- **Bon niveau** : accueil, solutions, contact, restaurant, admin login
- **Niveau moyen** : configurateur, about, landing-solo, coach
- **Zone la plus fragile** : configurateur et template landing-solo
- **Cause globale la plus probable** : trop de code, de styles et d'elements sont charges des l'ouverture, meme quand la page n'en a pas besoin

4. Constats confirmes

4.1 Conversion et confiance

C1. Bug confirme sur le formulaire pre-session landing-solo

Constat confirme :

- le champ email visible principal est lie a `responses.email`
- la logique de validation/sauvegarde peut attendre `responses.clientEmail` pour le template landing-solo

Consequences metier :

- l'utilisateur peut penser avoir rempli le bon champ
- la validation peut juger l'email incomplet
- l'enregistrement peut partir avec une valeur vide ou incoherente
- le parcours de vente peut etre freine ou casse

Preuves :

- le composant `FormulairePrÃ©Session.tsx` utilise a la fois `email` et `clientEmail`
- la logique de mapping specifique landing-solo force ensuite `responses.clientEmail`

Cause possible :

- deux conventions de nommage coexistent dans le meme composant
- le formulaire a grandi par ajouts successifs au lieu d'avoir un modele unique par template

Methode de resolution :

1. definir une seule cle email de reference par template
2. aligner champ visible, validation, sauvegarde et persistence sur cette meme cle
3. ajouter un test simple sur le cas landing-solo

Impact :

- **eleve**

Guide dev debutant :

Probleme explique simplement :

- le formulaire ne parle pas toujours du meme email
- l'utilisateur remplit un champ visible
- mais le code ne lit pas toujours cette meme valeur ensuite

Pourquoi c'est un vrai bug :

- un formulaire doit utiliser la meme information du debut a la fin

- ici, le champ visible, la validation et la sauvegarde ne semblent pas totalement alignes

Etapes conseilles pour corriger :

1. ouvrir `FormulairePrÃ©Session.tsx`
2. chercher toutes les occurrences de `email` et `clientEmail`
3. identifier quelle cle doit etre la bonne pour `landing-solo`
4. aligner :

le champ visible

la condition de validation

l'objet envoye au service

la reprise eventuelle des donnees

1. faire un test manuel :

email rempli -> soumission autorisee

email vide -> soumission bloquee

1. refaire un test avec un autre template pour verifier qu'on n'a rien casse

Piege a eviter :

- corriger seulement la ligne du champ sans corriger la logique de sauvegarde

C2. La page de confirmation peut afficher un succes sans verification serveur fraiche

Constat confirme :

- la page `/confirmation` lit des parametres d'URL
- elle reconstruit aussi l'etat depuis `localStorage`
- elle peut donc afficher une confirmation sans relecture serveur immediate

Consequences metier :

- un utilisateur peut voir un message rassurant sans preuve recente cote serveur
- cela fragilise la confiance apres paiement ou commande
- cela complique la gestion des litiges ou des faux positifs

Preuves :

- `ConfirmationPage.tsx` lit `paymentId` , `orderNumber` , puis `pendingOrder` depuis `localStorage`

Cause possible :

- la page de confirmation a ete pensee comme ecran de restitution front plutot que comme ecran de verification

Methode de resolution :

1. recuperer la commande cote serveur ou via Supabase a partir d'un identifiant fiable
2. n'afficher le succes complet qu'apres verification reelle
3. basculer sinon vers un etat neutre ou "verification en cours"

Impact :

- **eleve**

Guide dev debutant :

Probleme explique simplement :

- la page affiche un resultat tres rassurant
- mais elle peut le faire a partir de donnees deja presentes dans le navigateur
- cela ne garantit pas que la commande a vraiment ete relue et verifiee

Pourquoi c'est important :

- une page de confirmation ne doit pas "supposer" qu'une commande est validee
- elle doit s'appuyer sur une source fiable

Etapas conseillees pour corriger :

1. repÃ©rer dans `ConfirmationPage.tsx` ce qui vient de l'URL
2. repÃ©rer ce qui vient du `localStorage`
3. repÃ©rer s'il existe deja une fonction fiable pour relire la commande
4. choisir un identifiant de commande de reference
5. charger la commande a partir de cet identifiant
6. afficher :

un etat de chargement

un succes seulement si la commande est verifiee

un message neutre ou d'erreur sinon

Piege a eviter :

- garder les memes textes tres affirmatifs avant que la verification reelle soit en place

C3. Le formulaire de contact depend surtout de Crisp cote client

Constat confirme :

- le formulaire tente un envoi vers Crisp en front
- si Crisp n'est pas charge, le fallback actuel est un `console.log`

Consequences metier :

- un lead peut etre perdu sans trace exploitable
- l'utilisateur peut croire que sa demande est partie alors qu'elle n'est pas stockee

Preuves :

- `ContactPage.tsx` envoie via `window.$crisp.push(...)`
- en absence de Crisp, les donnees partent dans la console

Cause possible :

- le chat a ete utilise comme canal principal de reception, sans vrai plan de secours metier

Methode de resolution :

1. brancher le formulaire sur une persistance fiable ou un envoi backend
2. garder Crisp comme canal de conversation, pas comme unique point d'entree
3. journaliser l'echec d'envoi et afficher un message honnete a l'utilisateur

Impact :

- **eleve**

Guide dev debutant :

Probleme explique simplement :

- le formulaire essaie surtout d'envoyer vers Crisp
- si Crisp ne repond pas ou ne charge pas, on n'a pas de vrai filet de securite

Pourquoi c'est important :

- un formulaire de contact doit fonctionner meme si un script tiers est bloque

Etapes conseillees pour corriger :

1. ouvrir `ContactPage.tsx`
2. reperer la fonction d'envoi actuelle
3. verifier le comportement si `window.$crisp` n'existe pas
4. choisir un envoi principal fiable
5. brancher ce canal principal avant Crisp

6. afficher un message de succes seulement si cet envoi principal reussit
7. afficher un message d'erreur clair sinon

Piege a eviter :

- garder `console.log` comme fallback metier

C4. Certains messages de parcours promettent plus que ce qui semble reellement automatise

Constat confirme :

- apres paiement, la page de confirmation promet une suite qui semble automatique
- dans le code lu, l'envoi d'email ou certaines actions aval paraissent encore incompletes ou manuelles

Consequences metier :

- risque de deception apres achat
- ecart entre promesse visible et fonctionnement reel
- confiance entamee si la suite n'arrive pas comme annonce

Preuves :

- `ConfirmationPage.tsx` mentionne un email de confirmation avec lien Calendly
- `AdminOrderDetail.tsx` contient encore un point de traitement non finalise autour de l'email

Cause possible :

- messages de reassurance rediges avant stabilisation complete du circuit de suivi

Methode de resolution :

1. n'afficher qu'une promesse que le produit tient vraiment aujourd'hui
2. parler d'un delai de recontact realiste tant que l'automatisation complete n'est pas en place
3. ne mentionner Calendly que si l'envoi automatique existe reellement

Impact :

- **moyen a eleve**

Guide dev debutant :

Probleme explique simplement :

- les textes de confirmation semblent dire que certaines actions sont automatiques
- mais le code montre que tout n'est pas encore totalement branche

Etapes conseillees pour corriger :

1. lister les promesses affichees a l'utilisateur
2. verifier dans le code si chaque promesse est vraiment tenue
3. si ce n'est pas le cas, corriger d'abord le texte
4. ensuite seulement, automatiser si c'est bien prevu produit

C5. Le parcours de contact et de commande manque parfois de clarte

Constat confirme ou probable selon le chemin :

- l'accueil pousse surtout vers la commande et le configurateur
- la page contact melange deux intentions : poser une question ou commander

Consequences metier :

- certains visiteurs peuvent hesiter sur le bon point d'entree
- le formulaire libre peut faire doublon avec le configurateur

Preuves :

- `Home.tsx` met surtout en avant `Commander un site` et `Configurer mon site`
- `ContactPage.tsx` propose a la fois configurateur et formulaire libre sur le meme ecran

Cause possible :

- objectifs commerciaux et objectifs de conversation reunis dans la meme page

Methode de resolution :

1. separer clairement `Commander un site` et `Poser une question`
2. ajouter un CTA contact plus visible sur l'accueil pour les visiteurs qui ne veulent pas encore configurer
3. clarifier le role de chaque entree dans le wording

Impact :

- **moyen**

C6. Le formulaire pre-session demande un effort important sans assez de reperes

Constat confirme :

- le formulaire est dense
- il ne rassure pas assez sur la duree, l'avancement ou la reprise
- en cas de lien invalide, l'utilisateur tombe vite sur une impasse

Consequences metier :

- abandon plus probable
- effort percu plus lourd que necessaire
- frustration en cas de lien casse

Preuves :

- `FormulairePrÃ@Session.tsx` parle surtout de prendre son temps
- le composant ne montre pas de repere clair de progression
- le message d'erreur `Lien invalide` reste sec et peu aidant

Cause possible :

- logique de collecte complete privilegiee sur l'accompagnement du parcours

Methode de resolution :

1. annoncer une duree indicative simple
2. afficher une progression ou des etapes
3. dire clairement si les reponses sont sauvegardees ou non
4. transformer `Lien invalide` en message de recuperation avec contact ou renvoi

Impact :

- **moyen a eleve**

4.2 Performance et structure

C4. Le site charge trop de code pour des pages simples

Constat confirme :

- la build actuelle produit un JS principal de **945.62 kB**
- elle produit aussi un CSS principal de **210.81 kB**
- toutes les grandes pages sont importees statiquement dans `App.tsx`

Consequences :

- pages simples penalisees par des morceaux de l'application qu'elles n'utilisent pas

- temps de chargement percu degrade sur mobile
- marge d'optimisation facile encore disponible

Preuves :

- `npm run build`
- `App.tsx` importe directement home, about, contact, solutions, configurateur, admin, templates et confirmations
- les rapports Lighthouse voient de **141 a 183 KiB de JavaScript inutile** et **22 a 31 KiB de CSS inutile**

Cause possible :

- absence de chargement differe par route
- socle CSS global trop large
- configurateur, admin et templates inclus trop tot dans le bundle initial

Methode de resolution :

1. passer les grosses pages en chargement differe
2. separer au minimum configurateur, admin, templates et parcours commande
3. reduire le CSS global au strict socle commun

Impact :

- **eleve**

C5. Le LCP est trop haut sur plusieurs pages clefs

Constat confirme :

- configurateur : **5.4 s**
- about : **5.3 s**
- template landing-solo : **5.7 s**
- template coach : **4.3 s**

Lecture metier :

- la page commence a apparaitre
- mais le contenu principal arrive trop tard pour donner une sensation de fluidite

Causes possibles :

- bundle initial trop large

- images hero ou visuels de carte trop lourds
- plusieurs ressources externes chargees trop tot

Methodes de resolution :

1. alléger et fractionner le chargement initial
2. optimiser les images hero et utiliser des tailles adaptees
3. mettre les visuels secondaires en chargement differe

Impact :

- **moyen a eleve**

C6. Certaines pages chargent des medias trop tot ou trop lourdement

Constat confirme :

- la page about s'appuie sur une image hero lourde
- home et solutions chargent tres tot plusieurs visuels de cartes
- le template landing-solo charge de nombreuses images externes des l'arrivee

Consequences :

- transfert reseau plus lourd que necessaire
- sentiment de lenteur plus marque sur mobile

Preuves :

- rapport `about-prod-lighthouse.json`
- rapport `template-landing-solo-prod-lighthouse.json`
- visuels appeles dans `AboutPage.tsx` , `Home.tsx` , `SolutionsPage.tsx` et `LandingSolo.tsx`

Cause possible :

- logique de maquette riche conservee telle quelle en production

Methode de resolution :

1. prevoir des versions plus petites des medias critiques
2. reserver le chargement prioritaire au seul visuel principal
3. mettre le reste en `loading="lazy"` ou equivalent

Impact :

- **moyen a eleve**

4.3 SEO et contenu technique

C7. Le titre de page est global et statique

Constat confirme :

- `index.html` definit un seul titre global
- aucune logique claire de titre par page n'a ete relevee dans le code lu

Consequences :

- pages moins distinctes pour Google
- onglets navigateur peu informatifs

Cause possible :

- base de site SPA mise en place sans couche SEO page par page

Methode de resolution :

1. definir un titre specifique par page
2. prioriser les pages business : accueil, solutions, templates, contact
3. prevoir une convention simple commune pour tous les titres

Impact :

- **moyen**

C8. Les pages n'ont pas de meta description dediee

Constat confirme :

- aucune meta description exploitable n'a ete relevee dans `index.html`
- les scores SEO restent bloques autour de **82-83**

Consequences :

- extraits Google moins maitrisables
- message marketing moins clair en recherche

Cause possible :

- couche SEO minimale ou absente

Methode de resolution :

1. ajouter une description par page
2. harmoniser le ton et la promesse commerciale
3. preparer aussi les bases de partage social dans le meme mouvement

Impact :

- **moyen**

C9. Le maillage SEO public reste trop faible ou parfois incoherent

Constat confirme :

- les routes templates existent bien
- mais l'acces depuis les pages principales passe souvent par des boutons ou des comportements peu favorables au crawl
- le footer pointe aussi vers des chemins qui ne correspondent plus proprement a l'arborescence actuelle

Consequences :

- pages templates moins faciles a decouvrir pour Google
- maillage interne moins robuste
- impression de structure pas totalement stabilisee

Preuves :

- `SolutionsPage.tsx` s'appuie sur des ouvertures de demo plutot que sur un vrai maillage de liens
- `Footer.tsx` renvoie encore vers `/templates` et `/projects` alors que l'application redirige ou ne sert pas ces chemins comme pages business pleines

Cause possible :

- priorite donnee a la demonstration visuelle et aux modales plutot qu'au maillage de pages d'acquisition

Methode de resolution :

1. ajouter de vrais liens vers les templates publics depuis l'accueil, solutions et footer
2. supprimer ou corriger les liens de footer devenus ambigus
3. clarifier quelles pages doivent servir a l'acquisition SEO

Impact :

- **moyen a eleve**

C10. Le socle SEO technique est incomplet

Constat confirme :

- je n'ai pas releve de `robots.txt` , `sitemap.xml` , canonical, Open Graph, Twitter Cards ou donnees structurees dans le socle public actuel

Consequences :

- indexation moins robuste
- partage social moins maitrise
- signaux techniques faibles pour les moteurs

Cause possible :

- socle Vite/SPA garde proche du minimum technique

Methode de resolution :

1. ajouter `robots.txt` et `sitemap.xml`
2. ajouter canonical et metas sociales sur les pages publiques
3. prevoir au minimum un schema `Organization` et des schemas orientes service sur les pages business

Impact :

- **moyen**

4.4 Accessibilite

C11. Des problemes de contraste sont deja confirmes

Constat confirme :

- Lighthouse remonte un contraste insuffisant sur :
- `/configurator`
- `/templates/landing-solo`
- `/admin/login`

Consequences :

- lecture plus difficile
- acces degrade pour certains utilisateurs

Cause possible :

- palette visuelle pensee d'abord pour l'apparence, pas assez verifiee pour la lisibilite

Methode de resolution :

1. corriger les couples texte/fond les plus faibles
2. figer une palette conforme pour les textes principaux, secondaires et etats focus
3. verifier les memes composants sur plusieurs pages

Impact :

- **moyen**

C12. Les formulaires et la structure de page restent incomplets pour l'accessibilite

Constat confirme :

- le formulaire de contact utilise des champs sans labels relies
- plusieurs pages n'ont pas de zone principale `<main>`
- certains niveaux de titres sont incohérents

Consequences :

- navigation plus difficile au lecteur d'ecran
- parcours clavier moins fluides
- comprehension structurelle plus fragile

Preuves :

- `ContactPage.tsx` affiche des `input` et `textarea` appuyes surtout sur des placeholders
- les rapports Lighthouse remontent l'absence de zone principale sur plusieurs pages
- le configurateur et solutions montrent une hierarchie de titres perfectible

Cause possible :

- structure semantique ajoutee apres la mise en forme visuelle, au lieu d'etre pensee des le depart

Methode de resolution :

1. ajouter des `label` relies a chaque champ critique
2. encapsuler le contenu principal de chaque page dans un `<main>`
3. reprendre une hierarchie simple `h1 > h2 > h3`

Impact :

- **moyen a eleve**

C13. Plusieurs elements interactifs et modales restent peu accessibles au clavier

Constat confirme :

- certaines cartes cliquables reposent sur des `div` avec `onClick`
- plusieurs modales ne montrent pas clairement une gestion de focus et de role accessible

Consequences :

- parcours plus difficiles sans souris
- risque de piege de navigation dans les modales

Preuves :

- composants de cartes templates et portfolio
- `TemplateModal.tsx` , `OnboardingModal.tsx` , `ShareSessionModal.tsx` , `PreviewModal.tsx`

Cause possible :

- composants rendus interactifs apres coup, d'abord pour la vue

Methode de resolution :

1. remplacer les zones cliquables par de vrais `button` ou `a` quand c'est possible
2. ajouter `role="dialog"` , `aria-modal` , focus d'ouverture, retention du focus et restitution du focus a la fermeture

Impact :

- **moyen a eleve**

4.5 Qualite technique

C14. La dette technique reste elevee autour du configurateur et des formulaires

Constat confirme :

- le lint remonte **220 problemes**
- dont **197 erreurs** et **23 warnings**
- le gros du volume se concentre autour de composants deja sensibles

Zones les plus exposees :

- `ConfiguratorPage.tsx`
- `ConfigurationCaptureModalAdaptive.tsx`
- `FormulairePrÃ©Session.tsx`
- composants de paiement, preview et services associes

Consequences :

- corrections plus lentes
- risque de regression plus fort
- maintenance plus couteuse sur les parcours business

Cause possible :

- composant monolithique
- usage de `any`
- effets React fragiles
- accumulation de logique metier, visuelle et reseau au meme endroit

Methode de resolution :

1. decouper le configurateur en sous-modules
2. traiter en premier les types et les zones a impact business
3. ajouter quelques tests cibles avant les gros refactors

Impact :

- **moyen a eleve**

Guide dev debutant :

Probleme explique simplement :

- les gros fichiers font beaucoup de choses en meme temps
- ils contiennent beaucoup de `any` et plusieurs warnings de logique React
- cela rend les bugs plus durs a corriger proprement

Etapes conseillees pour corriger sans se perdre :

1. ne pas essayer de tout nettoyer d'un coup
2. cibler d'abord les composants lies aux parcours business
3. traiter en premier :

les types de donnees critiques

les `useEffect` fragiles

les erreurs liees aux soumissions et confirmations

1. relancer le lint regulierement
2. valider une petite amelioration stable avant de passer a la suivante

4.6 Securite et protection des donnees

C15. Une commande peut probablement etre validee cote client sans verification serveur suffisante

Constat confirme :

- des composants front semblent pouvoir marquer une commande comme payee sans preuve serveur visible dans le parcours lu

Consequences :

- risque critique de faux statut de paiement
- risque metier et financier

Preuves :

- `OrderValidation.tsx`
- logique de mise a jour cote client relayee vers `supabase.ts`

Cause possible :

- logique de confiance trop placee dans le navigateur

Methode de resolution :

1. deplacer la validation paiement et marketplace dans un backend ou une Edge Function
2. verifier le paiement cote serveur
3. ne mettre a jour `payment_status` qu'apres validation reelle

Impact :

- **critique**

Guide dev debutant :

Probleme explique simplement :

- le navigateur de l'utilisateur ne doit jamais etre l'autorite finale sur le paiement
- s'il peut lui-meme faire passer une commande a `paid` , c'est un gros risque

Etapas conseillees pour corriger :

1. trouver le moment exact ou `payment_status` passe a `paid`
2. verifier si ce changement part directement du front
3. creer ou utiliser un point serveur de verification
4. faire la verification du paiement a cet endroit
5. ne renvoyer au front qu'un resultat deja verifie

Piege a eviter :

- ajouter juste une verification cosmetique dans le front

C16. Des donnees client et identifiants de paiement transitent ou restent dans le navigateur

Constat confirme :

- le parcours utilise `localStorage`
- certains identifiants remontent dans l'URL ou dans l'etat front

Consequences :

- confidentialite plus fragile
- exposition inutile de donnees en cas de partage d'URL, machine partagee ou capture d'ecran

Cause possible :

- parcours simplifie par persistance locale et reconstruction de l'etat cote client

Methode de resolution :

1. retirer les donnees sensibles du `localStorage`
2. supprimer les identifiants de paiement de l'URL quand ils ne sont pas indispensables
3. basculer vers des identifiants serveur courts ou des etats de session ephemeres

Impact :

- **moyen a eleve**

Guide dev debutant :

Probleme explique simplement :

- plus on laisse de donnees sensibles dans l'URL ou le `localStorage` , plus on augmente le risque d'exposition

Etapes conseillees pour corriger :

1. lister ce qui est stocke dans le navigateur
2. supprimer ce qui n'a pas besoin d'y rester
3. remplacer par un identifiant plus court ou une lecture serveur
4. verifier ensuite les parcours de retour apres paiement

C17. Les assets client et certaines lectures de commande meritent une revue urgente des policies

Constat confirme ou probable selon l'environnement :

- les images client semblent converties en URLs publiques
- la lecture de commande `/order/:id` pourrait etre trop permissive si certaines policies documentees ont ete appliquees telles quelles

Consequences :

- risque de fuite de donnees client
- acces trop large a des informations de commande

Preuves :

- `supabase.ts`
- `AdminOrderDetail.tsx`
- `OrderConfirmationPage.tsx`
- documentation Supabase du projet

Cause possible :

- choix de simplicite pour faciliter l'affichage admin ou la consultation de commande

Methode de resolution :

1. passer les buckets sensibles en prive
2. utiliser des URLs signees courtes
3. verifier toutes les policies `SELECT USING (true)` ou `FOR ALL USING (true)`

Impact :

- **eleve**

Guide dev debutant :

Probleme explique simplement :

- une policy trop ouverte peut laisser lire des donnees qu'on ne voulait pas rendre publiques

Etapes conseillees pour corriger :

1. lister les tables et buckets sensibles
2. ouvrir les fichiers SQL et les policies en place
3. chercher les acces trop larges
4. fermer ces acces
5. retester lecture commande, images et sessions avec un compte non autorise

4.7 Coherence visuelle et qualite percue

C18. La promesse visuelle premium n'est pas tenue partout de facon homogene

Constat confirme :

- certaines pages suivent bien le socle bleu nuit / or
- d'autres ecrans glissent vers des gradients violets, des effets visuels et des composants qui sortent du systeme principal

Consequences :

- produit percu comme moins maitrise
- credibilite premium affaiblie

Preuves :

- `tokens.css`
- `AdminLogin.module.css`
- `ShareSessionModal.module.css`
- `ConfiguratorPage.module.css`

Cause possible :

- plusieurs couches UI ajoutees a des moments differents avec leurs propres codes visuels

Methode de resolution :

1. figer une palette prioritaire
2. remplacer les couleurs hors systeme par des variables communes
3. verifier les composants sensibles sur admin, modales et configurateur

Impact :

- **moyen**

C19. Le systeme de design existe mais il est partiellement contourné

Constat confirme :

- les tokens et variables existent
- mais plusieurs pages utilisent encore des conventions, alias ou familles typo qui ne semblent pas totalement alignés

Conséquences :

- impression de produit inachevé
- maintenance visuelle plus difficile

Preuves :

- écarts de variables entre `Home.tsx` et `tokens.css`
- déclaration de plusieurs familles de polices avec import ou usage incomplet

Cause possible :

- coexistence d'anciens noms et d'un nouveau système de design

Méthode de résolution :

1. garder un seul vocabulaire de tokens
2. limiter l'interface à un petit nombre de familles typographiques réelles
3. supprimer progressivement les alias et styles locaux incohérents

Impact :

- **moyen**

5. Hypotheses probables

H1. Le configurateur porte trop de responsabilités dans un seul écran

Probable, car :

- le fichier `ConfiguratorPage.tsx` fait environ **6729 lignes**
- il mélange logique métier, preview, sessions, personnalisation, données et parcours de commande

Effets probables :

- bugs difficiles a isoler
- corrections qui se marchent dessus
- lenteurs plus difficiles a reduire durablement

Methode de resolution probable :

- decoupage par etape ou domaine fonctionnel
- activation du temps reel et des appels externes seulement quand utile

H2. La strategie de chargement globale penalise toute la vitrine

Probable, car :

- `App.tsx` importe toutes les grosses pages des le depart
- le bundle final reste trop gros pour un site vitrine

Effets probables :

- pages simples penalisees par la complexite du configurateur et de l'admin

Methode de resolution probable :

- chargement differe par route
- feuilles de styles plus locales

H3. Plusieurs parcours critiques ont ete penses d'abord cote front

Probable, car :

- confirmation basee sur URL + `localStorage`
- contact tres dependant de Crisp
- plusieurs parcours semblent reconstruire l'etat dans l'interface
- certaines validations et lectures sensibles semblent encore trop proches du navigateur

Effets probables :

- confiance fragile
- faux positifs
- pertes silencieuses de donnees

H4. L'experience a grandi par ajouts successifs plus que par harmonisation globale

Probable, car :

- plusieurs parcours gardent des conventions de donnees, de styles et de messages differentes
- les retours design, accessibilite, SEO et qualite convergent vers une meme idee : le socle existe, mais il n'est pas encore applique partout de facon stable

Effets probables :

- experience moins rassurante qu'attendu
- ecarts plus visibles sur les zones secondaires ou recentes

Methode de resolution probable :

- harmoniser apres priorisation plutot que corriger ecran par ecran au hasard

Methode de resolution probable :

- reintroduire des points de verification serveur ou de persistence fiable

6. Points a verifier avec de vraies donnees

- dashboard admin apres authentication reelle
- page `/order/:id` avec une vraie commande
- confirmation de paiement de bout en bout
- formulaire pre-session avec ecriture reelle en base
- session collaborative `/configurator/session/:sessionId`
- comportement exact des politiques et droits Supabase

7. Plan de correction coherent

Lot 1. Securiser les parcours metier

Objectif :

fiabiliser ce qui touche a la confiance et a la conversion.

Actions :

1. corriger le mapping email du pre-session landing-solo
2. verifier la commande cote serveur avant d'afficher une confirmation pleine
3. connecter le formulaire contact a une persistence fiable
4. garder Crisp en soutien, pas en dependance unique

Lot 2. Alléger l'application

Objectif :

améliorer les performances sans casser la cohérence produit.

Actions :

1. charger les grosses pages à la demande
2. isoler `configurator`, `admin`, `templates` et `order` en chunks séparés
3. réduire les styles globaux non indispensables
4. optimiser les médias hero et différer les visuels secondaires

Lot 3. Relever le niveau SEO

Objectif :

donner à chaque page un rôle clair pour Google et pour l'utilisateur.

Actions :

1. ajouter un titre par page
2. ajouter une meta description par page
3. définir un socle commun : title, description, canonical, partage social
4. prioriser accueil, solutions, templates et contact

Lot 4. Corriger l'accessibilité visible

Objectif :

uniformiser la lisibilité et la navigation.

Actions :

1. corriger les contrastes faibles
2. vérifier les états focus et la lisibilité des textes secondaires
3. reprendre les formulaires critiques avec une grille simple de vérification

Lot 5. Réduire la dette technique

Objectif :

rendre les prochaines corrections plus simples et plus sûres.

Actions :

1. découper le configurateur

2. supprimer progressivement les `any` sur les zones critiques
3. stabiliser les `useEffect` fragiles
4. ajouter quelques tests cibles sur pre-session, confirmation et contact

8. Priorisation finale

A faire tout de suite

- corriger le bug email du formulaire pre-session
- fiabiliser la page de confirmation
- securiser le formulaire contact

A faire juste apres

- chargement differe par route
- titres et meta descriptions par page
- correction des contrastes

A faire ensuite

- refactor du configurateur
- reduction progressive de la dette lint/type
- tests reels avec donnees valides

9. Conclusion simple

Le site a deja une base credible et presentable.

Le vrai sujet n'est pas de tout refaire.

Le sujet est de **fiabiliser ce qui compte**, puis de **retirer le poids inutile** qui freine le reste.

La strategie la plus coherente reste :

1. **securiser la conversion**
2. **alleger le chargement**
3. **poser un vrai socle SEO**
4. **stabiliser le configurateur**

Avec ces 4 axes, on ameliore en meme temps :

- la confiance utilisateur
- la vitesse percue
- la visibilite SEO
- la capacite a faire evoluer l'outil sans casse

10. Preuves principales

- rapport de reference : `reports/consolidated/2026-04-06-audit-complet-site.md`
- rapports Lighthouse de production dans `reports/evidence/`
- build production locale validee
- lint local execute
- lecture ciblee du code sur les parcours critiques
- consolidation des retours des agents specialises